TapIn — Smart Attendance System

Full Project Documentation

1. Project Overview

Project Title: TapIn — Smart Attendance System **Version:** 1.0.0 **Platform:** Android Mobile Application (Flutter) **Backend:** Supabase (Cloud PostgreSQL + Authentication + Edge Functions) **Hardware:** NodeMCU ESP8266 + RC522 RFID Reader Module

TapIn is an IoT-integrated smart attendance management system designed for university use. A student taps their RFID card on a NodeMCU device installed in the classroom. The device reads the card, sends the unique RFID identifier to a Supabase Edge Function over HTTPS, and the system automatically records the attendance in a cloud database. Students and admins view this data in real time through this Flutter mobile app.

2. Technology Stack

Layer	Technology	Purpose
Mobile App	Flutter (Dart)	Cross-platform mobile application
Backend/Database	Supabase	Auth, PostgreSQL database, Edge Functions
IoT Hardware	NodeMCU ESP8266	WiFi microcontroller to scan RFID
RFID Reader	RC522 (MFRC522)	Reads 13.56 MHz student RFID cards
State Management	Flutter setState	Theme and UI state
Local Storage	shared_preferences	Persist dark/light mode setting
PDF Generation	pdf + printing	Generate and share attendance PDFs
Date Formatting	intl	Format dates and times in the UI

3. Hardware — NodeMCU and RFID System

3.1 Code File Location

```
assets/nodemcucode/nodemcucode.ino
```

3.2 Hardware Components

Component	Model	Role
Microcontroller	NodeMCU ESP8266	WiFi-enabled Arduino-compatible board

Component	Model	Role
RFID Reader	RC522 (MFRC522 library)	Reads 13.56 MHz RFID/NFC cards
RFID Card	Standard ISO 14443A	Each student has one with a unique UID

3.3 Physical Pin Connections (NodeMCU to RC522)

RC522 Pin	NodeMCU GPIO	NodeMCU Board Label
SDA (SS)	GPIO 15	D8 (defined in code as SS_PIN)
RST	GPIO 0	D3 (defined in code as RST_PIN)
MOSI	GPIO 13	D7 (hardware SPI default)
MISO	GPIO 12	D6 (hardware SPI default)
SCK	GPIO 14	D5 (hardware SPI default)
GND	GND	GND
3.3V	3.3V	3V3

Note: MOSI, MISO, SCK are the default ESP8266 hardware SPI pins and are automatically used by the SPI library. Only SS_PIN (D8) and RST_PIN (D3) are explicitly defined in code.

3.4 Required Arduino Libraries

- ESP8266WiFi.h — Connect NodeMCU to WiFi network
- ESP8266HTTPClient.h — Make HTTP POST requests
- WiFiClientSecure.h — Handle HTTPS/SSL connections
- SPI.h — SPI bus communication for RC522
- MFRC522.h — Control the RC522 RFID reader hardware
- ArduinoJson.h — Parse JSON responses from Supabase

3.5 NodeMCU Code Flow (Step by Step)

setup() function:

- Serial.begin(115200) — opens serial monitor for debugging
- SPI.begin() — initializes the SPI bus
- rfid.PCD_Init() — initializes the RC522 RFID module
- WiFi.begin(ssid, password) — starts WiFi connection (ssid: "naim")
- Waits in while loop until WL_CONNECTED, prints "." every 500ms
- Prints IP address to Serial Monitor when connected

loop() function:

- Calls rfid.PICC_IsNewCardPresent() — checks if a new RFID card is near
- If no card: delay(100) and return (keeps polling)
- Calls rfid.PICC_ReadCardSerial() — reads the card's serial data
- Iterates through rfid.uid.uidByte array, converts each byte to 2-char hex string

- Calls uid.toUpperCase() — example result: "EBACC836"
- Calls sendToSupabase(uid) to transmit the UID
- Calls rfid.PICC_HaltA() and rfid.PCD_StopCrypto1() to release the card
- delay(3000) — 3 second cooldown prevents double-recording the same tap

sendToSupabase(String uid) function:

- Checks WiFi.status() == WL_CONNECTED before proceeding
- Creates WiFiClientSecure and calls client.setInsecure() to skip SSL cert verification
- Creates HTTPClient, calls http.begin(client, functionUrl)
- Adds headers: Content-Type: application/json and apikey:
- Builds body string: {"rfid_uid":"EBACC836"}
- Calls http.POST(body) — sends HTTP POST to Edge Function
- If HTTP code > 0: reads response string, parses JSON using ArduinoJson
- Extracts: success (bool), message (string), student (string) from JSON
- Prints result to Serial Monitor
- Calls http.end() to free resources

3.6 Supabase Edge Function Details

- URL: https://nqmzpjaiphcfrnnlxhxv.supabase.co/functions/v1/mark-attendance
- Method: HTTP POST
- Request body: { "rfid_uid": "<CARD_UID>" }
- Response body: { "success": true/false, "message": "...", "student": "" }
- The Edge Function (Deno/TypeScript) looks up the student by rfid_uid in the users table, checks the active class_schedule, and inserts a record into attendance_logs.

---

4. Supabase Backend

4.1 Project Credentials

- Project URL: https://nqmzpjaiphcfrnnlxhxv.supabase.co
- Anon Key: sb_publishable_z2KxRFk5y0UQM0kNxsIJRQ_dVwFg2XN
- Auth Provider: Supabase built-in email/password (GoTrue)

4.2 Database Schema — 5 Tables

Table 1: users Stores all user profiles (students and admins).

Column	Type	Notes
id	UUID PK	Matches Supabase Auth user ID
name	Text	Full name
university_id	Text	Roll number e.g. 2101001
email	Text	Login email
role	Text	'student' or 'admin'

Column	Type	Notes
department	Text	e.g. 'IRE'
section	Text	e.g. '2021-22'
lab_group	Text	'G1' or 'G2' (auto-assigned)
phone_number	Text	Optional
rfid_uid	Text	Physical card UID e.g. 'EBACC836'

Table 2: subjects Academic subjects/courses.

Column	Type	Notes
id	UUID PK	Unique subject ID
name	Text	Full name e.g. 'Mobile Platform For IoT Devices'
code	Text	Short code e.g. 'IRE-401'
department	Text	Department code
section	Text	Academic session

Table 3: class_schedules Weekly class timetable.

Column	Type	Notes
id	UUID PK	Unique schedule ID
subject_id	UUID FK	References subjects.id
department	Text	Department code
section	Text	Academic session
day_name	Text	e.g. 'Sunday', 'Monday'
start_time	Time	e.g. 09:00:00
end_time	Time	e.g. 10:30:00
lab_group	Text	'all' for theory, 'G1'/'G2' for lab
is_active	Boolean	Whether schedule is active

Table 4: attendance_logs Core attendance records (one row per student per class).

Column	Type	Notes
id	UUID PK	Unique record ID
student_id	UUID FK	References users.id
subject_id	UUID FK	References subjects.id

Column	Type	Notes
date	Date	e.g. 2026-08-12
entry_time	Timestamptz	RFID tap-in time
exit_time	Timestamptz	RFID tap-out time (optional)
status	Text	'present', 'absent', or 'condoned'

Table 5: leave_applications Student leave requests.

Column	Type	Notes
id	UUID PK	Unique application ID
student_id	UUID FK	References users.id
subject_id	UUID FK	References subjects.id
leave_date	Date	Requested leave date
reason	Text	Student's explanation
leave_type	Text	'medical', 'general', or 'emergency'
status	Text	'pending', 'approved', or 'rejected'
admin_note	Text	Optional admin comment
created_at	Timestamptz	Auto-generated

5. Flutter App Structure

5.1 Directory Layout

```
lib/
├── main.dart
├── core/
│   ├── constants/app_constants.dart
│   └── theme/app_theme.dart
└── presentation/
    └── screens/
        ├── auth/
        │   ├── splash_screen.dart
        │   ├── login_screen.dart
        │   └── register_screen.dart
        ├── student/
        │   └── student_home.dart   (5 tabs, ~1500 lines)
        └── admin/
            └── admin_home.dart     (6 tabs, ~2087 lines)
```

5.2 main.dart

Class TapInApp (StatefulWidget) — root of the app.

main() function:

1. WidgetsFlutterBinding.ensureInitialized()
2. Supabase.initialize(url, anonKey)
3. prefs.getBool('isDarkMode') ?? false
4. runApp(TapInApp(isDarkMode: saved))

_TapInAppState:

- _isDarkMode: single source of truth for app theme
- toggleTheme(bool value): setState + SharedPreferences save
- Builds MaterialApp with themeMode, lightTheme, darkTheme, home: SplashScreen

5.3 app_constants.dart

Constant	Value
supabaseUrl	https://nqmzpjaiphcfrnnlxhxv.supabase.co
supabaseAnonKey	sb_publishable_z2KxRFk5y0UQM0kNxsIJRQ_dVwFg2XN
appName	'TapIn'
appTagline	'Smart Attendance System'
roleStudent	'student'
roleAdmin	'admin'
department	'IRE'
section	'2021-22'
minAttendancePercent	90.0
attendanceMark	30
entryWindowMinutes	15
exitWindowMinutes	15

5.4 app_theme.dart — Color Palette

Token	Hex	Usage
primary	#2563EB	Buttons, active nav, links
accent	#06B6D4	Lab indicators, gradients
success	#10B981	Present status
warning	#F59E0B	Condoned/pending status
error	#EF4444	Absent/error status

Token	Hex	Usage
lightBg	#F8FAFC	Light mode background
lightSurface	#FFFFFF	Light mode cards
lightText	#0F172A	Light mode text
lightTextSecondary	#64748B	Light mode subtitles
lightBorder	#E2E8F0	Light mode borders
darkBg	#0A0F1E	Dark mode background
darkSurface	#111827	Dark mode nav bar
darkCard	#1E2640	Dark mode card background
darkText	#F1F5F9	Dark mode text
darkTextSecondary	#94A3B8	Dark mode subtitles
darkBorder	#1E293B	Dark mode borders

---

6. Screen-by-Screen Documentation

6.1 splash_screen.dart (SplashScreen)

Type: StatefulWidget with TickerProviderStateMixin

Animations:

- _logoController (900ms): _logoScale (0.5->1.0 elasticOut), _logoOpacity (0->1 easeIn)
- _textController (700ms, starts after logo): _textOpacity (0->1), _textSlide (Offset(0,0.3)->Offset.zero)
- 3 loading dots: TweenAnimationBuilder per dot, staggered 200ms each

_navigate():

1. Future.delayed(2500ms)
2. Check Supabase.instance.client.auth.currentSession
3. No session -> LoginScreen (FadeTransition 600ms)
4. Session exists -> query users table for role
5. role=='admin' -> AdminHome, else -> StudentHome

Visual: 100x100 gradient container (radius 28) with fingerprint icon (52px), "TapIn" text (38px weight 800), "Smart Attendance System" subtitle (14px), 3 animated dots.

6.2 login_screen.dart (LoginScreen)

Type: StatefulWidget

State: _emailController, _passwordController, _isLoading, _obscurePassword, _errorMessage

_login() logic:

1. Validate fields not empty
2. auth.signInWithPassword(email, password)
3. Query users table for role
4. role=='admin' -> pushReplacement AdminHome
5. else -> pushReplacement StudentHome
6. Catch AuthException -> show _errorMessage

UI: Fingerprint icon with gradient, email/password fields, show/hide toggle, error banner, Sign In button with loading spinner, Register link, Light/Dark mode toggle pill.

6.3 register_screen.dart (RegisterScreen)

Type: StatefulWidget

Fields: name, university_id, email, password (min 6 chars), phone (optional)

_getLabGroup(universityId):

- Parses last 3 digits of university ID
- <= 25 -> 'G1', > 25 -> 'G2'

_register() logic:

1. Validate required fields + password length
2. auth.signUp(email, password) -> creates Auth user
3. Insert into users table: id, name, university_id, email, role='student', department='IRE', section='2021-22', lab_group (auto), phone_number
4. Show success SnackBar -> navigate to LoginScreen

Note: Admin accounts must be created directly in Supabase dashboard with role='admin'.

6.4 student_home.dart (StudentHome) — 5 Tabs

StudentHome (StatefulWidget):

- _currentIndex: active tab (0-4)
- _userData: logged-in student's users table record
- _loadUserData(): queries users table, passes data to all tabs

Bottom Nav: Home(0), Attendance(1), History(2), Leave(3), Profile(4)

Tab 0: _DashboardTab

_loadData():

- class_schedules query: filter by dept, section, today's day_name, is_active=true, lab_group filter (all OR student's group), join with subjects, order by start_time
- attendance_logs query: filter by student_id, count present/absent/total, calculate percentage

UI:

- "Hello, [Name] wave" header with university_id and lab group
- Gradient summary card: overall percentage (large text), circular progress indicator (checkmark if >=90%, else exclamation), present/absent chips
- Today's classes list: left color bar (blue=theory, cyan=lab), subject code + name, time range, theory/lab badge
- "No classes today!" message when list is empty
- Pull-to-refresh (RefreshIndicator)

Tab 1: _AttendanceTab

_loadData():

- subjects query: filter by dept and section
- attendance_logs query: filter by student_id
- For each subject: filter logs by subject_id, count present/absent/total, calculate percentage

UI: List of subject cards with subject code, name, percentage badge (green>=90%, amber>=75%, red<75%), colored linear progress bar, present/absent/total chips.

Tab 2: _HistoryTab

_loadData():

- attendance_logs query: join subjects, filter by student_id, order date descending, limit 50

UI: List of attendance records showing status icon (check/info/cancel), subject code, entry->exit time (hh:mm a format), date, PRESENT/ABSENT/CONDONED status badge.

Tab 3: _LeaveTab

_loadLeaves():

- leave_applications query: join subjects, filter by student_id, order created_at descending

_applyLeave() (FloatingActionButton -> bottom sheet):

1. Fetch subjects for dropdown
2. Show modal with: subject dropdown, leave type dropdown (Medical/General/Emergency), date picker, reason text field (3 lines)
3. Insert into leave_applications with status='pending'
4. Refresh list

UI: Leave cards showing subject code, PENDING/APPROVED/REJECTED status badge, reason, admin_note (if present), date and type.

Tab 4: _ProfileTab (StatelessWidget)

Shows: name, university_id, email, department, section, lab_group, phone Settings: Dark/Light mode toggle, Sign Out (signOut + pushAndRemoveUntil to LoginScreen)

6.5 admin_home.dart (AdminHome) — 6 Tabs

AdminHome (StatefulWidget): Bottom Nav: Dashboard(0), Students(1), Attendance(2), Leave(3), Schedule(4), Profile(5)

Tab 0: _AdminDashboardTab

_loadData() — 4 queries:

1. users: count students in dept/section
2. attendance_logs: today's records (joined with users)
3. class_schedules: today's classes (joined with subjects, is_active=true)
4. leave_applications: pending count

UI:

- "Admin Panel" title + today's date (EEEE, dd MMM)
- 2x2 stats grid: Total Students (blue), Today Present (green), Pending Leaves (amber), Today Classes (cyan)
- Today's schedule list with same card format as student dashboard
- Pull-to-refresh

Tab 1: _AdminStudentsTab

_loadStudents(): query users where role='student', dept='IRE', section='2021-22', ordered by university_id

_search(query): client-side filter by name or university_id

_showStudentDetails(student): opens _StudentDetailSheet as ModalBottomSheet

UI: Search bar + student list (gradient avatar with initial, name, university ID, lab group badge)

_StudentDetailSheet:

- Loads attendance stats from attendance_logs for the specific student
- Shows: attendance%, present, absent, condoned, total
- Status banner: "Attendance mark secured (30/30)" or "not secured (0/30)" based on >=90%
- Contains _RfidAssignWidget

_RfidAssignWidget:

- Shows current rfid_uid from student record
- TextField for entering RFID UID (auto-uppercase)
- Save button: .from('users').update({'rfid_uid': rfid}).eq('id', studentId)
- This is the critical admin operation to link a physical card to a student account

Tab 2: _AdminAttendanceTab

State: _selectedDate (defaults to today), _logs list

_loadData(): attendance_logs joined with users and subjects, filtered by selected date, ordered by entry_time

_pickDate(): Flutter DatePicker (range 2024 to today), reloads on selection

_updateStatus(logId, status): updates attendance_logs.status field

UI: Date picker row + list of records per date. Each record: student name/ID, subject code, entry time, status as PopupMenuButton (Present/Absent/Condoned selectable).

Tab 3: _AdminLeaveTab

State: _filter ('pending' default), _leaves list

_loadLeaves(): leave_applications joined with users and subjects, filtered by _filter status

Approval flow:

- Approve: update leave status to 'approved' + update attendance_logs to 'condoned'
- Reject: update leave status to 'rejected'
- Optional admin_note field

UI: Filter tabs (Pending/Approved/Rejected) + leave cards showing student info, subject, date, reason, admin_note, Approve/Reject buttons (visible only on Pending tab).

Tab 4: _AdminScheduleTab

State: _selectedDay (today's day name), _schedules list

_days list: ['Saturday','Sunday','Monday','Tuesday','Wednesday','Thursday'] (Bangladesh academic week)

_loadSchedules(): class_schedules joined with subjects, filtered by day, dept, section, is_active=true, ordered by start_time

UI: Horizontal scrollable day buttons + class list. Class cards: left color bar (theory=blue, lab=cyan), subject code/name/time, "Lab G1"/"Lab G2"/"Theory" badge.

Tab 5: _AdminProfileTab (StatelessWidget)

UI: Admin icon with gradient, "Administrator" label, "IRE Department • 2021-22" subtitle. Settings: theme toggle, sign out.

---

7. Complete App Navigation Flow

```
App Launch
    |
    v
SplashScreen (2.5s animated)
    |
    |-- No session --> LoginScreen
    |                       |
    |                       |-- Sign In (admin) --> AdminHome (6 tabs)
    |                       |-- Sign In (student) --> StudentHome (5 tabs)
    |                       |-- "Register" link --> RegisterScreen
    |                                                   |
    |                       <-- Registration success ------|
    |
    |-- Session + admin role --> AdminHome (6 tabs)
    |-- Session + student role --> StudentHome (5 tabs)
```

All screens: Sign Out -> signOut() -> pushAndRemoveUntil -> LoginScreen

---

8. Dark Mode System

1. App start: main() reads 'isDarkMode' from SharedPreferences
2. _TapInAppState._isDarkMode initialized from stored value
3. MaterialApp uses themeMode: _isDarkMode ? ThemeMode.dark : ThemeMode.light
4. toggleTheme(bool) passed to all screens via constructor
5. All screens detect theme with: Theme.of(context).brightness == Brightness.dark
6. User taps theme toggle -> onThemeToggle(!isDark) called -> toggleTheme() -> setState() -> MaterialApp rebuilds -> instant theme switch
7. New value saved to SharedPreferences for next launch

---

9. All Dart Classes Summary

Class	File	Type	Purpose
TapInApp	main.dart	StatefulWidget	Root app widget
_TapInAppState	main.dart	State	Theme management
SplashScreen	splash_screen.dart	StatefulWidget	Animated launch screen
_SplashScreenState	splash_screen.dart	State	Animations + auth check
LoginScreen	login_screen.dart	StatefulWidget	Login form
_LoginScreenState	login_screen.dart	State	Login logic + routing
RegisterScreen	register_screen.dart	StatefulWidget	Registration form
_RegisterScreenState	register_screen.dart	State	Register logic + lab assignment
StudentHome	student_home.dart	StatefulWidget	Student app shell
_StudentHomeState	student_home.dart	State	Tab management + user data
_DashboardTab	student_home.dart	StatefulWidget	Today's classes + stats
_DashboardTabState	student_home.dart	State	Supabase data loading
_AttendanceTab	student_home.dart	StatefulWidget	Per-subject attendance %
_AttendanceTabState	student_home.dart	State	Subject stats computation
_HistoryTab	student_home.dart	StatefulWidget	Attendance history list
_HistoryTabState	student_home.dart	State	Load 50 recent records
_LeaveTab	student_home.dart	StatefulWidget	Leave applications list
_LeaveTabState	student_home.dart	State	Load + submit leaves

Class	File	Type	Purpose
_ProfileTab	student_home.dart	StatelessWidget	Profile + settings
AdminHome	admin_home.dart	StatefulWidget	Admin app shell
_AdminHomeState	admin_home.dart	State	Tab management
_AdminDashboardTab	admin_home.dart	StatefulWidget	Overview stats + schedule
_AdminDashboardTabState	admin_home.dart	State	4 parallel queries
_AdminStudentsTab	admin_home.dart	StatefulWidget	Student list + search
_AdminStudentsTabState	admin_home.dart	State	Load + filter students
_StudentDetailSheet	admin_home.dart	StatefulWidget	Student stats bottom sheet
_StudentDetailSheetState	admin_home.dart	State	Load individual stats
_RfidAssignWidget	admin_home.dart	StatefulWidget	RFID card assignment
_RfidAssignWidgetState	admin_home.dart	State	Save rfid_uid to DB
_AdminAttendanceTab	admin_home.dart	StatefulWidget	Date-filtered attendance
_AdminAttendanceTabState	admin_home.dart	State	Date picker + status update
_AdminLeaveTab	admin_home.dart	StatefulWidget	Leave approval management
_AdminLeaveTabState	admin_home.dart	State	Filter + approve/reject
_AdminScheduleTab	admin_home.dart	StatefulWidget	Weekly timetable view
_AdminScheduleTabState	admin_home.dart	State	Day selector + schedule load
_AdminProfileTab	admin_home.dart	StatelessWidget	Admin settings
AppTheme	app_theme.dart	Plain class	Colors + ThemeData objects
AppConstants	app_constants.dart	Plain class	All constant values

10. Packages and Dependencies

Package	Version	Purpose
supabase_flutter	^2.8.4	Full Supabase client (auth, db, realtime)
shared_preferences	^2.3.3	Persist dark mode setting locally
pdf	^3.11.1	Generate PDF attendance reports
printing	^5.13.1	Print or share PDFs via device
intl	^0.19.0	DateFormat class for date/time display
uuid	^4.5.1	Generate UUIDs
cupertino_icons	^1.0.8	iOS-style icon set

Package	Version	Purpose
flutter_launcher_icons	^0.13.1	Generate app icon from assets/icon.png

11. Android Configuration

File: android/app/src/main/AndroidManifest.xml

- App label: "TapIn Attendance"
- App icon: custom icon from assets/icon.png (generated by flutter_launcher_icons)
- Required permission: android.permission.INTERNET (for all Supabase HTTPS calls)

12. Business Rules

1. Lab Group Assignment: Based on last 3 digits of university ID. Digits <= 25 assign G1, digits > 25 assign G2.
2. Attendance Threshold: Minimum 90% attendance is required (AppConstants.minAttendancePercent). Students at or above this receive 30/30 attendance marks.
3. RFID Linking: Admin MUST assign an RFID card UID to a student profile before that student's card taps are recognized by the NodeMCU. Without this, the Edge Function cannot find the student.
4. Condoned Status: When admin approves a leave application, the corresponding attendance_log record is updated to 'condoned'. Condoned absences count as valid attendance for the percentage calculation.
5. Bangladesh Academic Week: Classes run Saturday through Thursday. Friday is excluded. The AdminScheduleTab _days list reflects this: ['Saturday','Sunday','Monday','Tuesday','Wednesday','Thursday'].
6. 3-Second Scan Cooldown: NodeMCU waits 3 seconds after each RFID scan before scanning again. This prevents double-recording if a student holds their card too long.
7. Fixed Department Scope: Version 1.0.0 is hard-coded for department='IRE' and section='2021-22'. All database queries use these hardcoded values. Multi-department support would require future refactoring.
8. Admin Account Creation: Admin users cannot register through the app. They must be created manually in the Supabase dashboard by: (a) creating an Auth user, (b) inserting a record in the users table with role='admin' and the matching Auth UUID as the id.

Document generated: August 2026 | TapIn Attendance System v1.0.0